2026 International Conference on Next-Gen Quantum and Advanced Computing: Algorithms, Security, and Beyond (NQComp) | 979-8-3315-5935-9/26/$31.00 ©2026 IEEE | DOI: 10.1109/NQComp68334.2026.11497643

# Cloud Security & Auto-Remediation Framework Using AWS Native Services

Nidhindev Valiyedath
*REVA Academy for Corporate Excellence, REVA University,*
Bengaluru, India
nidhin.CS11@race.reva.edu.in

Pradyumn Nand
*Eventbrite*
Hyderabad, India
pnand@eventbrite.com

Shinu Abhi
*REVA Academy for Corporate Excellence, REVA University,*
Bengaluru, India
shinuabhi@reva.edu.in

**Abstract— Modern cloud estates built on Amazon Web Services (AWS) evolve rapidly across multiple accounts and regions, delivering business value but also creating security challenges such as configuration drift, where the actual security state diverges from the intended policy. Periodic audits and ticket-based remediation introduce long exposure windows and inconsistent compliance evidence, leaving enterprises vulnerable. This research addresses these gaps by designing and evaluating an AWS-native, risk-informed auto-remediation framework that transforms static policies into continuous, automated controls. The framework targets recurring high-impact misconfigurations and reduces delays between detection and corrective action. Objectives include minimising time to remediate, improving compliance coverage, and ensuring audit-ready evidence while maintaining service stability for sensitive operations. The methodology follows a design–build–evaluate cycle, mapping policies to industry benchmarks and encoding them as AWS Config rules. AWS Config provides continuous detection, Amazon EventBridge routes findings, and decision logic classifies risks, invoking automated remediation with AWS Lambda and Systems Manager for low-risk issues, while approval workflows manage high-risk cases such as encrypting root Elastic block stores volumes. Structured evidence is emitted to CloudWatch, and compliance status is updated in AWS Config. Controlled testing showed the prototype reduced remediation time from hours to minutes, achieved near real-time compliance updates, and maintained traceability from detection to resolution. Approval-gated Elastic block store encryption balanced automation with human oversight, reducing risks without delaying remediation. The analysis further indicates that preventive controls in continuous integration/continuous deployment pipelines and organization-wide encryption defaults can minimise recurrence. Overall, this work demonstrates a scalable AWS-native framework that delivers constant compliance and significantly shortens exposure windows across a multi-account environment.**

***Keywords—Cloud Security, AWS, Configuration Drift, Auto Remediation, AWS Config, EventBridge, Lambda***

## I. INTRODUCTION

Public cloud environments, especially those built on AWS, evolve rapidly as teams deploy frequent changes across accounts and regions, integrate third-party services, and update infrastructure in parallel. While this speed enhances delivery, it also leads to configuration drift—where actual states diverge from intended policies due to ad-hoc console edits, emergency patches, or inconsistent process maturity. Traditional security practices such as periodic audits, manual reviews, and ticket-based remediation are slow, resource intensive, and leave prolonged exposure windows exploitable by adversaries. Common high-risk misconfigurations, including public S3 buckets [1],[2] overly permissive security groups, wildcard Identity and access Management (IAM) policies, unencrypted storage, and disabled logging, persist despite well-documented standards. To address these challenges, this study introduces a risk-informed, AWS-native auto-remediation framework that transforms static compliance rules into dynamic, self-healing guardrails. Leveraging AWS Config for detection, *EventBridge* for event routing, and Lambda with Systems Manager Automation for enforcement, the framework applies a risk matrix to determine whether remediation should be automated or require human approval for sensitive actions like root Elastic block store encryption. Evidence generation is prioritised through structured logging, compliance status updates, and tagged artifacts, ensuring continuous audit readiness. By aligning with Center for Internet Security AWS Foundations, National Institute of Standards and Technology Special Publication 800-53, and International Organization for Standardization / International Electrotechnical Commission 27001, the system achieves measurable improvements in mean time to remediate (Mean time to response) compliance coverage, and assurance traceability. The remaining sections III and IV cover related work and benchmarks methodology and framework design implementation details analysis and results and discussion of risks, limitations, and future scope.

## II. LITERATURE REVIEW

An in-depth examination of the existing literature and a comprehensive overview of the state-of-the-art in cloud security are provided in this section. Some of the major challenges identified include configuration drift, recurring high-impact misconfigurations, long remediation delays, and the lack of audit-ready evidence in multi-account AWS estate.

### *A. Configuration Drift, Misconfigurations, and Benchmarks*

Cloud security research consistently identifies configuration drift and human error as persistent drivers of cloud risk. Industry breach analyses confirm that misconfigurations such as publicly exposed S3 buckets, insecure Kubernetes manifests, and over-permissive IAM roles remain among the most common error categories linked to cloud breaches [3],[4]. This underscores the need for continuous detection and remediation rather than reliance on periodic audits

Standards and benchmarks form the baseline for policy definition in AWS. The Center for Internet Security AWS Foundations Benchmark prescribes configuration requirements across identity, logging, networking, and data protection domains, while AWS Security Hub integrates these requirements for continuous posture evaluation [5],[6]. National Institute of Standards and Technology Special Publication 800-53 Rev. 5 provides a comprehensive control catalogue, while Special Publication 800-128 emphasizes translating policies into enforceable configuration states [7]. International Organisation for Standardisation/International Electrotechnical Commission 27001:2022 frames these

technical controls within an Information Security Management System model, reinforcing risk-based control selection and continuous improvement [8]. Together, these frameworks motivate automated mapping of detection and remediation mechanisms to normative controls with evidence suitable for audit.

### *B. Continuous Monitoring and Risk-Informed Decision-Making*

National Institute of Standards and Technology Special Publication 800-137 defines Information Security Continuous Monitoring as maintaining continuous awareness of assets, vulnerabilities [9] and control performance. In cloud environments, event-driven telemetry enables near real-time detection and enforcement, replacing periodic audits. Frameworks like National Institute of Standards and Technology Special Publication 800-30 and ISO 31000 guide risk-based decisions, automating low/medium-risk remediations while routing high-risk cases for timed human approval to ensure stability.[10],[11], [12]

### *C. AWS-Native Remediation and Policy-as-Code*

Vendor guidance and academic research converge on AWS-native automation pipelines. AWS Config evaluates compliance via managed and custom rules; *EventBridge* routes findings; and automated enforcement is achieved through Systems Manager Automation runbooks or AWS Lambda functions [13],[14]. AWS also provides reference architectures, such as the "Automated Security Response on AWS" solution, to operationalise remediation playbooks (e.g., closing open security groups, restricting S3 bucket access) across multi-account environments. These approaches embody idempotent and auditable pipelines that materially shorten mean time to remediate (Mean time to response).

Policy-as-Code frameworks extend this pipeline upstream to prevent misconfigurations at source. Research on Infrastructure-as-Code highlights recurring "security smells" in Terraform, Helm charts, and CloudFormation templates [11],[15]. Tools such as Open Policy Agent and Cloud Custodian encode controls as versioned policies, enabling both pre-merge checks in continuous integration / continuous deployment pipelines and runtime enforcement actions. This dual build-time and run-time control loop helps reduce recurrence of drift.

### *D. Conflicting Findings and Research Gaps*

While industry studies quantify misconfigurations and vendor playbooks describe remediation patterns, there is limited peer-reviewed evidence on the measurable outcomes of auto-remediation frameworks—particularly their impact on MEAN TIME TO RESPONSE, false positive rates, and long-term service stability across multi-account AWS estates. Some studies caution that aggressive automation may introduce reliability risks without proper guardrails, reinforcing the need for approval workflows and rollback strategies. Emerging research also explores automation assisted by large language models, though consensus on operational readiness remains unclear.

### *E. Positioning*

This research addresses the identified gaps by proposing a measurable, risk-aware, evidence-rich remediation pipeline. The framework maps Center for Internet Security, National Institute of Standards and Technology Special Publication, and International Organization for Standardization / International Electrotechnical Commission 27001 controls to AWS Config rules, uses *EventBridge* for routing, and applies enforcement through *Lambda* and *Systems Manager* Automation. audit-ready evidence is generated via CloudWatch and AWS Config compliance updates, directly addressing literature gaps around assurance and traceability. The solution aims to reduce Mean time to response for critical misconfigurations, maintain service stability via risk-informed decisioning, and provide reusable patterns for scalable, AWS-native compliance automation.

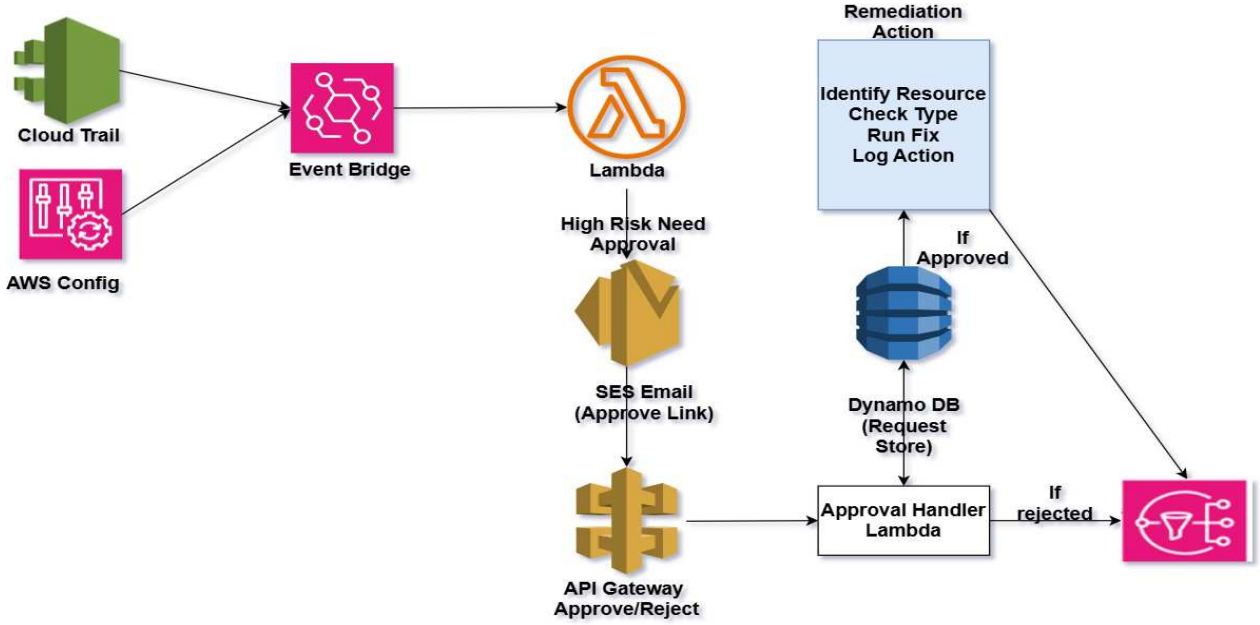


Fig. 1. High-Level Architectural Diagram (Structured Approach)

## III. METHODOLOGY AND DESIGN

The proposed methodology follows a design–build–evaluate–iterate approach tailored for AWS security engineering, ensuring each stage is evidence-driven. It begins with scoping and policy mapping, aligning misconfigurations such as public S3 access, open security groups, *Identity Access Management wildcards*, and unencrypted Amazon Elastic Block Store to benchmarks like CIS AWS Foundations, NIST SP 800-53, and ISO/IEC 27001. Success metrics target an MTTR ≤10 minutes and ≥50% compliance coverage. A canary AWS account supports baseline detection and dry-run remediation using CloudWatch, AWS Config, and SNS/SES for latency measurement. Detection leverages AWS Config, *CloudTrail*, and *Security Hub*, while *EventBridge* applies a likelihood-impact matrix for automated or approval-based routing. *Lambda* and *Systems Manager runbooks* perform least-privilege remediations with snapshots, TTL-based approvals, and rollback safeguards. As shown in Fig.1 The *event-driven, serverless pipeline (Detect–Route–Decide–Act–Validate)* ensures modularity and traceability through *EventBridge* routing, risk-based decisions, domain-specific remediators, approval validation, evidence logging, and compliance updates in AWS Config.

TABLE I. Tools Mapped to the Problem

| **Problem Pattern** | **Tooling (How it solves it)** |
|---|---|
| Public S3 access | AWS Config S3 rules detect → EventBridge route → Lambda/SSM enforces Block Public Access / fixes policy → Config compliance update + CloudWatch evidence |
| Open Security Groups | Config SG rule detects → EventBridge → Lambda removes 0.0.0.0/0 on sensitive ports → evidence logged |
| Identity and access Management (IAM) | Custom Config rule flags Action:* or Resource:* → Lambda suggests or applies scoped permissions; exceptions via allow-list → evidence and tags |
| Unencrypted EBS | Config encryption rule detects →EventBridge → Approval (high-risk) via API Gateway/DynamoDB/SES →Lambda/SSM encrypts snapshot + re-attach pattern (safe path) → Config re-evaluation confirms compliance |

### *A. Data Collection and Analysis*

The framework's evaluation is based on defined metrics, data sources, and comparative methods to measure efficiency and reliability. Key indicators include Mean Time to Remediate (MTTR) at p50 and p90, compliance coverage, approval latency, false-positive rate, and mean time between recurrences. Data is gathered from AWS Config, CloudWatch, DynamoDB, SNS, and Terraform to ensure traceability and completeness. Success criteria target MTTR ≤10 minutes for critical issues, ≥95% compliance coverage, and zero high-severity incidents. These structured metrics and analyses demonstrate measurable improvements in the overall cloud security.

## IV. IMPLEMENTATION

### *A. Common Pattern Across Modules*

Each auto-remediation module follows a four-stage pattern: *Detect → Route → Act → Evidence*. AWS Config identifies non-compliant resources, *EventBridge* routes findings to the right Lambda function, and the Lambda applies safe, idempotent fixes. Evidence is logged in CloudWatch, and AWS Config is updated via the *PutEvaluations* API to mark compliance. Each Lambda runs with a least-privilege IAM role, encrypted CloudWatch logs, and SNS notifications containing correlation and rollback details. Environment variables act as feature flags to switch between Enabled/Disabled and Dry_Run/Enforce modes, ensuring flexibility and control.

### *B. Auto-Remediation Modules*

The modular architecture enables the development of specific remediators for common AWS misconfigurations. For example, the S3 remediation module addresses the risk of public access to storage buckets. The Lambda handler begins by taking a snapshot of the current bucket policy to preserve state for potential rollback. It then enforces AWS Block Public Access settings, prunes policy statements containing Principal:"*", and finally calls *PutEvaluations* to update the compliance status in AWS Config as Compliant. The full implementation code for this module is maintained in the research's GitHub repository.

### *C. Security Group Remediation*

The Security Group remediation module fixes high-risk misconfigurations where admin ports like SSH (22) and RDP (3389) are exposed to the public internet. It uses an AWS Lambda function to detect and remove ingress rules allowing unrestricted access (0.0.0.0/0) to these ports. After remediation, the function updates AWS Config through the *PutEvaluations* API to mark the resource as compliant. The design ensures idempotence and reversibility using pre-change snapshots of the security group. All remediation evidence, including logs and compliance updates, is stored in CloudWatch and AWS Config timelines.

### *D.* IAM *Remediation*

The IAM remediation module addresses overly permissive inline policies containing wildcards like Action:"*" or Resource:"*", which pose high security risks. Implemented via AWS Lambda, it scans role policies and replaces detected wildcards with least-privilege templates stored in the /lambda/policies/ directory. After successful remediation, the function updates AWS Config using the *PutEvaluations* API to mark the resource as Compliant. A feature flag (*Auto_Fix=false*) allows switching to approval-only mode in test or canary environments to prevent unintended changes. All remediation evidence, including policy updates and compliance logs, is captured in CloudWatch for audit and traceability.

### *E. Relational Database Service Remediation*

The Relational Database Service remediation module mitigates the risk of publicly accessible databases by enforcing the configuration parameter.

*PubliclyAccessible*=false for supported Relational Database Service engines. Implemented through an AWS Lambda function, it invokes the *ModifyDBInstance* Application Programming Interface with Apply *Immediately=true* to apply changes instantly and polls until updates are confirmed. Once complete, the function reports compliance to AWS Config via the *PutEvaluations* API. The design ensures minimal downtime by limiting changes to engines supporting in-place updates, with all actions logged in CloudWatch and Config timelines. This AWS-native, idempotent approach provides a secure, auditable, and automated mechanism to maintain non-public Relational Database Service configurations.

### *F. ElasticBlock Store Encryption Remediation (Approval-Gated)*

The Elastic Block Store encryption remediation module enforces encryption at rest for Elastic Block Store volumes while prioritizing operational safety through a human approval workflow. When AWS Config flags a volume as Non-compliant, the first Lambda function triggers an approval request and emails authorised approvers with a signed review link. Upon approval via Application Programming Interface Gateway, the second Lambda function initiates encryption using AWS Systems Manager (SSM) or in-Lambda steps for smaller volumes. Compliance status and evidence are updated in AWS Config and shared with stakeholders for traceability. This design ensures secure, auditable, and rollback-safe encryption enforcement through AWS-native automation and controlled human oversight.

### *G. Systems Manager Automation (Optional but Recommended)*

To improve reliability and prevent Lambda timeouts in long-running encryption workflows, AWS Systems Manager *Systems Manager* Automation serves as a recommended orchestration layer. A custom document, such as Elastic block store Encrypt and Swap, manages the encryption process through sequential steps *snapshot* creation, volume encryption, instance stop/start, and validation. This AWS-native orchestration ensures rollback safety and controlled execution, reducing operational risk. The SSM document, version-controlled via Terraform, enhances observability and performance during encryption, especially for large root volumes.

## V. ANALYSIS AND RESULTS

The testing of the framework established functional correctness across all remediation modules and confirmed the ability of the system to deliver continuous detection, automated or approval-based remediation, and audit-ready evidence. Both quantitative metrics, such as Mean time to response distributions, false positives, and rollback rates, and qualitative aspects, including auditability, safety, and scalability, were evaluated in controlled test scenarios. Median Mean time to response trends were plotted and validated against the research's performance targets, while approval workflows and rollback safeguards, particularly for Elastic Block Store encryption, were assessed for operational safety and service continuity.

### *A. Outcome of the Methodology*

The event-driven pipeline of Detect → Route → Decide → Act → Validate operated seamlessly across all control categories. AWS Config identified findings, *EventBridge* routed them to handlers, and Lambda or Systems Manager Automation applied idempotent fixes while updating compliance status in CloudWatch. Elastic block store encryption followed an approval-first workflow using DynamoDB for time-bound tracking and explicit authorization. Quantitative analysis showed auto-remediated controls consistently meeting Mean time to response targets, with p50 under 10 minutes and p90 near thresholds. Rollbacks were rare, false positives minimal, and continuous tuning of *EventBridge* filters further improved accuracy and operational stability.

### *B. Control-by-Control Behavior*

The analysis of individual controls showed consistent and reliable remediation outcomes across all test cases. Security Group violations with open ports were quickly resolved by revoking 0.0.0.0/0 access, ensuring smooth Non-compliant to Compliant transitions. S3 public access fixes, achieved through Block Public Access and policy pruning, delivered rapid compliance with minimal recurrence. IAM wildcard and EC2 remediations were effective, maintaining least privilege and secure defaults with low latency. Elastic block store encryption and Relational database service exposure remediations followed approval-first workflows and automation through Systems Manager, ensuring zero downtime and verified instance health.

### *C. Reliability and Evidence Quality*

The framework's evidence pipeline was consistent and audit-ready. Structured JSON logging with stable fields enabled repeatable queries in CloudWatch Logs Insights, facilitating computation of Mean time to response, success rates, false positives, and approval latency. Evidence completeness met audit requirements by capturing timestamps for detection,

remediation, validation, approver identity (when relevant), and final compliance status. This comprehensive record established a chain of custody suitable for audits, regulatory reviews, and incident investigations.

### *D. Benchmark Comparison and Impact*

The automated remediation framework significantly reduced Mean Time to Response compared to manual, ticket-based processes, with the most notable gains in S3 and Security Group *remediations* where repetitive fixes were easily automated. Although Elastic Block Store encryption required approvals, it still outperformed manual workflows through standardized automation and predefined rollback mechanisms. Overall, the system minimized exposure windows, improved operational consistency, and reduced engineering effort while enforcing idempotent fixes and structured evidence collection to eliminate human variance and provide real-time visibility into approvals, rollbacks, and exceptions. The framework excelled through end-to-end automation, modular control design, and risk-aware approvals with rollback safeguards supported by a strong evidence trail. Its main limitations included dependency on AWS Config's cadence, IAM remediation noise, and variable EBS encryption times. Future enhancements should focus on CI/CD policy enforcement, contextual risk scoring, and expanded control coverage, while mitigating risks related to high event loads, API changes, and expired exceptions. Overall, the analysis confirms that the AWS-native, risk-informed auto-remediation framework meets its objectives by reducing exposure windows through continuous detection and response, achieving measurable MTTR improvements, and ensuring audit-ready, compliant operations in multi-account AWS environments.

## VI. CONCLUSIONS

The research achieved its objectives by developing an AWS-native, risk-informed auto-remediation framework utilizing Config*, EventBridge* , Lambda, DynamoDB, and CloudWatch. Its modular Detect–Route–Decide–Act–Validate pipeline ensured scalability, accountability, and evidence-based operation. The framework reduced exposure windows and achieved a Mean Time to Response (MTTR) under 10 minutes for frequent misconfigurations, while approval-gated workflows with rollback safeguards enabled secure automation. Automation significantly minimized remediation delays, with only minor latency in human-approved tasks such as Elastic Block Store encryption, which improved through on-call routing and automated low-risk approvals. Quantitatively, MTTR metrics remained within the 10-minute target across control categories, with minimal deviations linked to service latency or snapshot size. Regional performance variations and IAM policy complexity introduced some limitations, though these were mitigated through approval-only modes and preventive guardrails. Future work should enforce SLOs of MTTR p90 $\leq$ 10 minutes, maintain a false-positive rate $\leq$ 3%, and strengthen approval workflows, rollback mechanisms, and governance controls to ensure reliability and compliance at scale.

Future extensions include expanding the control library to cover additional use cases like Key Management Service key rotation, CloudTrail integrity checks, and Transport Layer Security enforcement using the same modular pattern. The framework can also integrate contextual risk scoring to decide dynamically between auto-remediation and approval-based actions. Finally, scaling to multi-account and multi-cloud environments through centralised governance or integration with Azure Policy and Google Cloud Platform Config Validator will enable unified security posture management.

.

## REFERENCES


[1] X. Wang, "Understanding Security Implications of Exposed Cloud Capabilities," in Proceedings of the USENIX Security Symposium, Anaheim, CA, USA, 2023.

[2] "There's a Hole in that Bucket: A Large-scale Analysis of Misconfigured S3 Buckets," research preprint [Online], 2019. Available: https://arxiv.org/abs/2508.14402

[3] A. Rahman, S. I. Shamim, D. B. Bose, and R. Pandita, "Security Misconfigurations in Open Source Kubernetes Manifests," ACM Transactions on Software Engineering and Methodology, vol. 32, no. 4, pp. 1–25, May 2023, doi: 10.1145/3579639.

[4] A.Rahman,"Different Kind of Smells: Security Smells in Infrastructure as Code Scripts," M.S. thesis, Dept. Comput. Sci., Univ. of Waterloo, Waterloo, ON, Canada, 2021.

[5] Amazon Web Services. (2023). AWS Security Hub documentation [Online].Available:https://docs.aws.amazon.com/securityhub/ Accessed on:oct.19,2025.

[6] Center for Internet Security, "CIS Amazon Web Services Foundations Benchmark" [Online], Accessed on:oct.19,2025. Available: https://www.cisecurity.org/cis-securesuite/cis-securesuite-membership-terms-of-use/

[7] Security and Privacy Controls for Information Systems and Organizations, NIST SP 800-53r5, 2020.

[8] Guide for Security-Focused Configuration Management of Information Systems, NIST SP 800-128, 2019.

[9] Information Security Continuous Monitoring (ISCM) for Federal Information Systems and Organizations, NIST SP 800-137, 2011..

[10] Assessing Information Security Continuous Monitoring (ISCM) Programs, NIST SP 800-137A, 2020.

[11] Guide for Conducting Risk Assessments, NIST SP 800-30r1, 2012.

[12] Risk Management-Guidelines, ISO 31000:2024, 2024..

[13] Amazon Web Services. (2023, May 15). Automated security response on AWS [Online]. Accessed on:oct.19,2025 Available: https://aws.amazon.com/solutions/implementations/automated-security-response-on-aws/

[14] Amazon Web Services. (2023, May 16). Automated security response on AWS – Implementation details [Online].Accessed on:oct.19,2025 Available:https://aws.amazon.com/solutions/implementations/automated-security-response-on-aws/.

[15] Cloud Native Computing Foundation. (2025). Open Policy Agent (OPA)[Online].Accessedon:oct.19,2025 Available:https://www.cncf.io/projects/cloud-custodian/